Thanks for waiting! Here's a **line-by-line explanation** of the **first two code cells** in your notebook:

☑ Code Cell 1

```
python
CopyEdit
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error, r2_score
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
```

Explanation:

- `import pandas as pd`: Imports **pandas** for data manipulation using DataFrames.
- `import numpy as np`: Imports **NumPy** for numerical operations.
- `import matplotlib.pyplot as plt`: Imports **Matplotlib** for data visualization (plots, graphs).
- `import seaborn as sns`: Imports **Seaborn**, a visualization library built on top of Matplotlib.
- `from sklearn.model_selection import train_test_split`: Imports function to **split dataset** into training and testing parts.
- `from sklearn.preprocessing import MinMaxScaler`: Imports a scaler to **normalize data** between 0 and 1.
- `from sklearn.metrics import mean_squared_error, r2_score`: Imports functions to **evaluate model performance**.
- `from tensorflow.keras.models import Sequential`: Imports Keras **Sequential model** (stacked linear layers).
- `from tensorflow.keras.layers import Dense`: Imports **Dense (fully connected)** layer for neural networks.

☑ Code Cell 2

```
python
CopyEdit
df = pd.read_csv("D:/BE Lab Practicals/Sem 8/DL/Lab
1/1_boston_housing.csv")
X = df.loc[:, df.columns != 'MEDV']
y = df.loc[:, df.columns == 'MEDV']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=123)
df
```

Explanation:

- `pd.read_csv(...)`: Reads the **Boston Housing dataset** from the given path into a pandas DataFrame.
- `X = df.loc[:, df.columns != 'MEDV']`: Selects **all columns except 'MEDV'** as features.
- `y = df.loc[:, df.columns == 'MEDV']`: Selects **only the 'MEDV' column** (target variable, i.e., median house value).
- `train_test_split(...)`: Splits data into training and testing sets (70% train, 30% test), `random_state=123` ensures repeatability.
- `df`: Displays the DataFrame in the notebook.

Would you like me to continue explaining the next few cells in the notebook?

4o

You said:

continue

ChatGPT said:

Great! Let's continue with the **next few cells** of your notebook.

☑ Code Cell 3

```
python
CopyEdit
scaler = MinMaxScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Explanation:

- `MinMaxScaler()` creates a **normalizer** that scales values to a range of `[0, 1]`.
- `scaler.fit_transform(X_train)`:
  - Fits the scaler **only on the training data** (to avoid data leakage).
  - Transforms the training data so that all feature values lie between 0 and 1.
- `scaler.transform(X_test)`:
  - Uses the **same scaler** (fitted to training data) to scale test data.

☑ Code Cell 4

```
python
CopyEdit
model = Sequential()
model.add(Dense(32, input_dim=X_train.shape[1], activation='relu'))
```

```
model.add(Dense(64, activation='relu'))
model.add(Dense(1))
```

Explanation:

- `model = Sequential()`: Initializes a **Sequential neural network model**.
- `model.add(Dense(32, input_dim=..., activation='relu'))`:
  - Adds a **Dense (fully connected) layer** with 32 neurons.
  - `input_dim=X_train.shape[1]` sets the number of input features.
  - `'relu'` is the activation function used (Rectified Linear Unit).
- `model.add(Dense(64, activation='relu'))`: Adds a second hidden layer with 64 neurons and ReLU activation.
- `model.add(Dense(1))`: Adds the output layer with 1 neuron (because this is a **regression task**, predicting a single value).

☑ Code Cell 5

```
python
CopyEdit
model.compile(optimizer='adam', loss='mse')
```

Explanation:

- `compile(...)`: Prepares the model for training.
  - `optimizer='adam'`: Uses the **Adam optimizer** (adaptive learning).
  - `loss='mse'`: Uses **Mean Squared Error** as the loss function (commonly used for regression problems).

☑ Code Cell 6

```
python
CopyEdit
history = model.fit(X_train_scaled, y_train, epochs=100, batch_size=32,
validation_split=0.2)
```

Explanation:

- `model.fit(...)`: Trains the model on the training data.
  - `X_train_scaled`: Scaled training features.
  - `y_train`: Training targets (house prices).
  - `epochs=100`: Train for 100 full passes over the dataset.
  - `batch_size=32`: Use mini-batches of 32 samples per training update.
  - `validation_split=0.2`: Use **20% of training data** as validation data (to monitor overfitting).
- `history`: Stores training and validation loss during training (can be used for plotting).

Would you like a plot of the training vs. validation loss next? Or should I continue with the next cells?

4o

You said:

continue

ChatGPT said:

Great! Let's continue with the remaining cells from your notebook.

☑ Code Cell 7

```
python
CopyEdit
loss = model.evaluate(X_test_scaled, y_test)
print("Test Loss:", loss)
```

Explanation:

- `model.evaluate(...)`: Tests the model on **unseen test data**.
  - `X_test_scaled`: Scaled input features from the test set.
  - `y_test`: True output values for the test set.
- `loss`: The Mean Squared Error (MSE) on the test set.
- `print(...)`: Displays how well the model performed on test data.

☑ Code Cell 8

```
python
CopyEdit
y_pred = model.predict(X_test_scaled)
```

Explanation:

- `model.predict(...)`: Uses the trained model to **make predictions** on the test set.
- `y_pred`: Contains the **predicted house prices** corresponding to `X_test_scaled`.

☑ Code Cell 9

```
python
CopyEdit
```

```
plt.scatter(y_test, y_pred)
plt.xlabel("Actual Prices")
plt.ylabel("Predicted Prices")
plt.title("Actual vs Predicted Prices")
plt.show()
```

Explanation:

- `plt.scatter(...)`: Plots a scatter plot comparing actual vs. predicted values.
- `xlabel(...)`, `ylabel(...)`: Labels the axes.
- `title(...)`: Sets a title for the plot.
- `plt.show()`: Displays the plot.

🔎 A perfect model would result in all points lying on a straight diagonal line (Actual = Predicted).

☑ Code Cell 10

```
python
CopyEdit
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
print("Mean Squared Error:", mse)
print("R^2 Score:", r2)
```

Explanation:

- `mean_squared_error(...)`: Calculates MSE between actual and predicted prices.
- `r2_score(...)`: Computes **R^2 Score**, which indicates how well the model explains the variability in the data.
  - $R^2 = 1$ → Perfect prediction
  - $R^2 = 0$ → No better than predicting the mean
- `print(...)`: Displays the metrics to assess model accuracy.